```
    #4
  node.children[c] ← new Node(false)
                  #5
  return addNewBranch (node.children[c],
tail)                                  #6
```

#1 Method `addNewKey` takes a trie node and the string key `s` for the new branch. As always, we assume that node is never `null`.
#2 Check if the string searched is empty: if it is, we added all the edges needed to the new branch, and current node is the last node in the path.
#3 Therefore, to complete the insertion, we just need to set current node to a key node.
#4 Since `s` is not the empty string, it can be broken into a *head* character, `c` (key first character of `s`) and a *tail*, the rest of the string.
#5 We add a new outgoing edge to current node, marked with character `c`; at the other end of the edge, we create a new empty node.
#6 We recursively add the characters in `tail` as a new branch of `children[c]`.

Similarly to what we did for search, we can prove that insert also takes `O(m)` amortized time, if the creation of a new node can be performed in constant time (which is the case if we use hash tables for edges, but not if we use plain arrays).

6.2.4 Remove

When it comes to removing a key from a trie, we are in the position to choose: we can go for an easier, cheaper